

Thesis Defense Game

Student Name: RenXuan

Student ID: ZY2557207

Date: 2026-06-11

Hosted App: https://cicisteady.github.io/assignment2_blog/thesis_defense_game.html

1. Project Overview

For Assignment 4, I built a playable web game called **Defend Your Thesis**. The concept is a one-minute survival-defense game where the player is a graduate student defending a thesis core and a committee gate from academic threats such as bugs, deadlines, and peer reviewers.

2. Functional Software

The application runs in a browser and is integrated into the existing GitHub Pages blog.

- Character selection page with 7 playable characters
- Canvas-based maze and real-time rendering
- Thesis core and committee gate defense mechanic
- Incoming humanoid Bugs, Deadlines, and Peer Reviewers
- Citation collectibles and score logic
- Keyboard controls with Arrow keys or WASD
- Mouse/touch movement by clicking or tapping the canvas
- One-minute timer, score, defended count, thesis integrity, rejection gate, and threat strength HUD
- Clear success/failure overlay and replay flow
- Defense objects with different rewards: Code Lab, Schedule Gate, and Committee Door
- Threats shrink after successful defense and grow stronger after failed defense
- Gate archers automatically shoot matching attackers from range
- Player sword rescue mechanic for close-combat emergencies

- Responsive layout for desktop and mobile screens

3. Game Design

The player moves through a maze, collects citations, and intercepts threats before they reach their target objects. Bugs attack the Code Lab, Deadlines attack the Schedule Gate, and Peer Reviewers attack the Committee Door. The round lasts 60 seconds. The player passes by surviving the full minute with thesis integrity above 0% and rejection gate below 100%.

If the player misses too many threats, those threat types become larger, faster, and more dangerous. When the rejection gate reaches 100%, the player is rejected at the door.

Each gate has an archer helper. The Code Lab archer shoots Bugs, the Schedule Gate archer shoots Deadlines, and the Committee Door archer shoots Peer Reviewers. If an attacker reaches an archer, the archer is pinned down and cannot shoot until the player uses the sword to rescue them.

Characters

Character	Role
Data Knight	Balanced defender
Citation Mage	Wider citation collection aura
Method Engineer	Strong and stable defender
Coffee Sprinter	Fastest movement
Algorithm Ninja	Quick interception
Lab Guardian	Durable thesis protection
Reviewer Whisperer	Bonus score against peer reviewers

Defense Objects

Threat	Target Object	Successful Defense	Failed Defense
Bugs	Code Lab	Debug Point repairs integrity; Code Lab archer helps shoot Bugs	Bugs grow stronger and can pin the archer
Deadlines	Schedule Gate	Time Buffer slows pressure; Schedule archer helps shoot Deadlines	Deadlines grow stronger and can pin the archer

Peer Reviewers	Committee Door	Review Point lowers rejection risk; Committee archer helps shoot Reviewers	Reviewers grow stronger and can pin the archer
----------------	----------------	--	--

4. Architecture

HTML page

- > shared site CSS
- > game JavaScript
- > Canvas game rendering
- > HUD and game state updates

The implementation uses `createState()`, `update(dt)`, `updateAllies()`, `updateArrows()`, `rescueAlly()`, `pathDirection()`, `draw()`, `endRound()`, and event listeners for controls.

5. AI-Assisted Development

AI helped plan the game framework, convert the assignment requirements into concrete features, and implement specific systems such as character selection, collision detection, pathfinding, score logic, archer helpers, arrow projectiles, sword rescue, threat growth, and responsive UI.

AI was most useful when the request was specific, such as asking for a tile-based collision system or a Canvas rendering loop. The final code still required manual verification, link checking, and deployment integration.

6. Technical Details

- **Rendering:** HTML Canvas with a 28 by 21 tile maze.
- **Canvas Size:** 896 by 640 pixels with responsive CSS scaling.
- **Collision:** Future-position wall checks with X/Y axis movement.
- **Pathfinding:** Threats move toward their target objects using tile-based route selection.
- **Helpers:** Gate archers shoot matching attackers; pinned archers require player sword rescue.
- **Loop:** `requestAnimationFrame` with delta-time updates.
- **Win/Fail:** Survive 60 seconds to pass; fail if integrity reaches 0% or rejection reaches 100%.

- **Controls:** Arrow keys, WASD, and pointer movement.

7. Verification

```
node --check site\assets\thesis_defense_game.js
```

The maze was checked as 21 rows by 28 columns, matching the 896 by 640 canvas when each tile is 32 pixels.

8. Reflection

This assignment showed the difference between an interesting prompt and a working software product. A stable game needs explicit state management, rendering, collision detection, pathfinding, controls, scoring, clear failure conditions, responsive layout, and deployment.

AI accelerated the implementation, but the developer still needed to define the goal, keep scope realistic, test the result, and prepare the final submission.

9. Conclusion

The final Assignment 4 submission is a functional hosted web game with character selection, one-minute gameplay, score logic, threat growth/weakening mechanics, keyboard and mouse controls, and documentation explaining AI-assisted development.